Artificial Intelligence for robotics

Part 6 : Multi-layer neural networks, implementation

Simon Gay

Introduction à l'Intelligence Artificielle

- **Reminder: network training: 3 steps**
  - Forward propagation: we calculate the output values of neurons, from input layer to output layer:

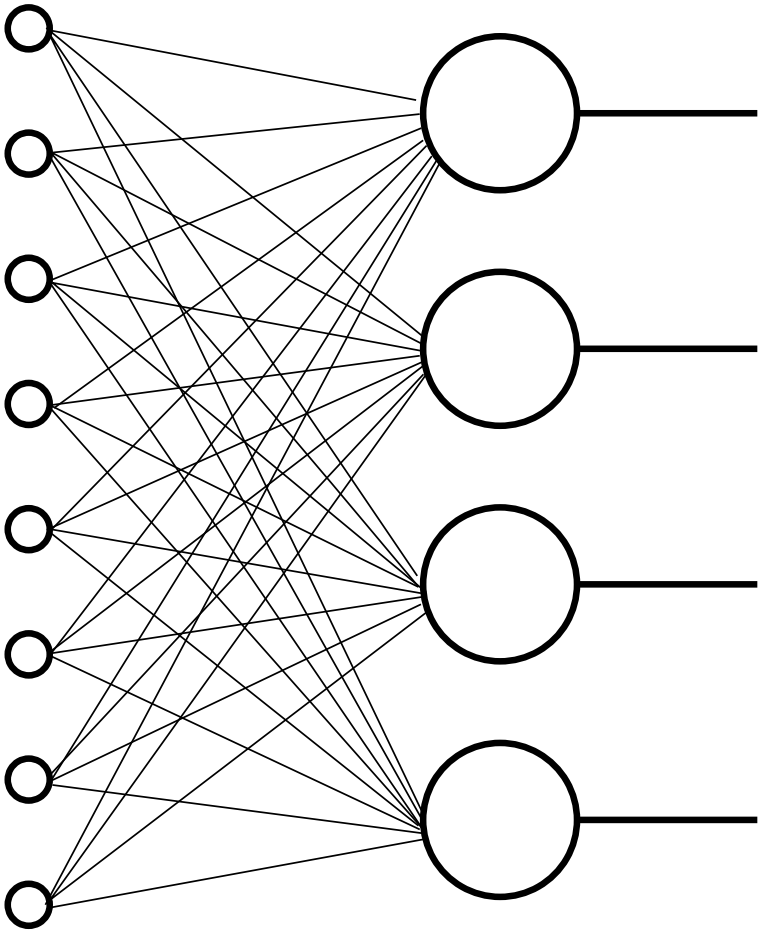
$$y_i^l = f\left(\sum_{k \in [0,n]} w_{ik}^l . x_k^l + b_i^l \right)$$

  - Backward propagation: we calculate the delta value of the output layer, then propagate deltas until the input layer
    - Last layer $\delta_i^L = f'_{(s_i^L)} . \left(r - y_i^L\right)$
    - Hidden layer $\delta_i^l = f'_{s_i^l} . \sum_{k \in [1,n]} \delta_k^{l+1} . w_{ki}^{l+1}$
  - Update of weights using deltas previously calculated

$$w_{ij}^l \Leftarrow w_{ij}^l + \alpha . x_j^l . \delta_i^l$$

$$b_i^l \Leftarrow b_i^l + \alpha . \delta_i^l$$

Artificial Intelligence for Robotics

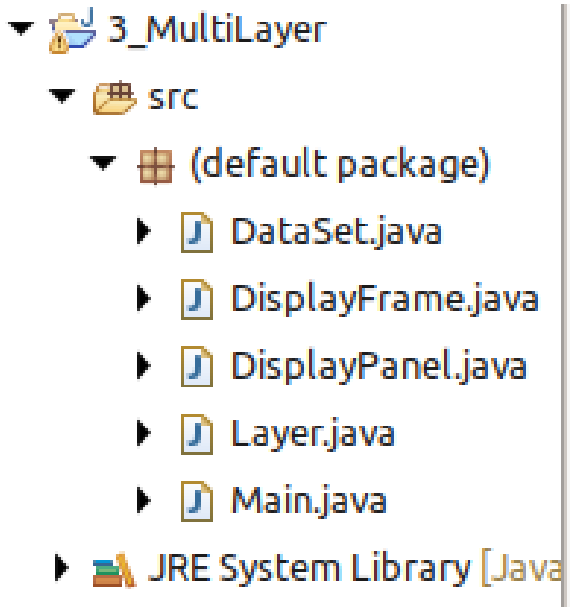


Let’s code !

Artificial Intelligence for Robotics

- **Open Eclipse on your workspace**

- Duplicate project 2_SingleLayerV2 and call the copy 3_MultiLayer
  - Right-click on ‘2_SingleLayerV2’ → copy → paste in ‘Package Explorer’ and rename



Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- First, we initialize weights with random values (-0,005 to 0,005):
  - Random function in Java : Math.random() → return a float value between 0 to 1 (excluded)

```
public Layer(int nb_input, int nb_output){
    weights=new float[nb_output][nb_input+1]; // +1 for bias
    results=new float[nb_output];
    deltas=new float[nb_output];

    for(int i = 0; i < nb_output; i++) {
        for(int j = 0; j < nb_input + 1; j++) {
            weights[i][j] = (float) ( Math.random() * 0.01 - 0.005);
        }
    }
}
```

Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- The *compute* function does not change:

```
public void compute(float[] input){
    for (int n=0;n<results.length;n++){ // each neuron
        results[n]=0; // reset sums
        for (int i=0;i<input.length;i++){
            results[n]+= weights[n][i]*input[i]; // /!\ index
        }
        results[n]+=weights[n][input.length]; // bias
        results[n]=activation(results[n]);
    }
}
```

Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- The *learn* function is divided into three functions, one for deltas of last layer, one for deltas of hidden layers, one for weights update:

$$\delta_i^L = f'_{(s_i^L)} \cdot \left(r - y_i^L\right)$$

$$\delta_i^l = f'_{s_i^l} \cdot \sum_{k \in [1,n]} \delta_k^{l+1} \cdot w_{ki}^{l+1}$$

$$w_{ij}^l \Leftarrow w_{ij}^l + \alpha . x_j^l . \delta_i^l$$

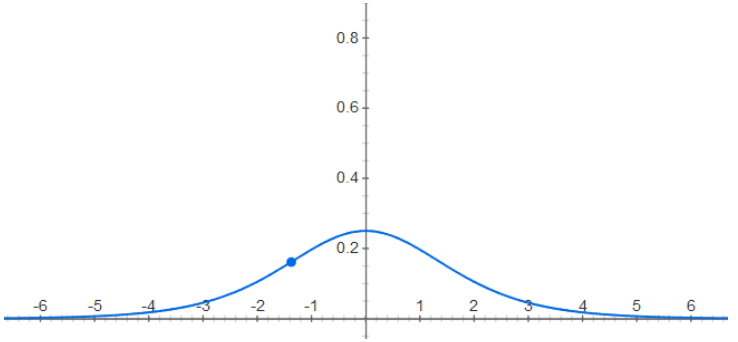
Introduction à l'Intelligence Artificielle

- **Reminder: case of Sigmoïd activation function**

  - With a sigmoïd activation function:

  - Derivative of sigmoïd function:

$$\sigma'(x) \quad = \quad \sigma(x) \ . \ (1-\sigma(x))$$



  - We already calculated σ during the *forward propagation*: they are the neuron's *y*
    - We thus obtain the following equation to calculate the deltas:

$$\delta_i^L \quad = \quad y_i^L \ . \ (1-y_i^L) \ . \ (r-y_i^L)$$

$$\delta_i^l \quad = \quad y_i^l \ . \ (1-y_i^l) \ . \ \sum_{k \in [1,n]} \delta_k^{l+1} \ . \ w_{ki}^{l+1}$$

Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- Add the setLastDeltas function:

$$\delta_i^L = f'_{(s_i^L)} \cdot (r - y_i^L) \qquad \delta_i^L = y_i^L \cdot (1 - y_i^L) \cdot (r - y_i^L)$$

```
public void setLastDeltas(int[] output) {
    sum_delta = 0;
    for(int i = 0; i < deltas.length; i++) {
        deltas[i] = results[i] * (1 - results[i]) * (output[i] - results[i]);

        // get the total error (only on last layer)
        sum_delta += Math.abs(deltas[i]);
    }
}
```

Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- Then, the setDeltas function for hidden layers:

$$\delta_i^l = f'_{s_i^l} \cdot \sum_{k \in [1,n]} \delta_k^{l+1} \cdot w_{ki}^{l+1} \qquad \delta_i^l = y_i^l \cdot (1 - y_i^l) \cdot \sum_{k \in [1,n]} \delta_k^{l+1} \cdot w_{ki}^{l+1}$$

- Here, we need to access the result of neurons in next layer, we will add a pointer to next layer

```
public void setDeltas(Layer next) {

    for(int i = 0; i < deltas.length; i++) {

        float sum=0; // sum of weighted deltas

        // add delta of k-th neuron in next layer, weighted by weights i->k
        for(int k = 0; k < next.weights.length; k++) {
            sum+= next.deltas[k] * next.weights[k][i]; // /!\ indexes
        }
        deltas[i]=  results[i]*(1-results[i]) * sum; // multiply with activation's derivative
    }
}
```

Artificial Intelligence for Robotics

- **Change the Layer Class to implement gradient descent**

- Finally, the update function for all layers:

$$w_{ij}^{l} \Leftarrow w_{ij}^{l} + \alpha . x_{j}^{l} . \delta_{i}^{l}$$

$$b_{i}^{l} \Leftarrow b_{i}^{l} + \alpha . \delta_{i}^{l}$$

- We just need to remove delta calculation and output parameter in function learn

```
public void learn(float[] input){

    // update weights
    for (int n=0;n<results.length;n++){
        for (int i=0;i<input.length;i++){
            weights[n][i]+= learnRate * deltas[n] * input[i];
        }
        weights[n][input.length]+= learnRate * deltas[n]; // bias
    }
}
```

- **The new version of Layer is complete !**

Artificial Intelligence for Robotics

- **Change the learning and evaluation process**

- We need to change the network structure in **Main** class to create a 2 layers network
- We define variables for layers' size
- The network is now a vector of layers

```
public int epoch=0;
public int test=0;

public static int nb_layer1 = 20;           // size of first layer
public static int nb_layer2 = 10;           // nb of classes (size of last layer)

//-----------------
private DisplayFrame display;       // display panel


//-----------------
public Layer[] layers;              // network

//////////////
public Main(){

    // initialize structures
    layers=new Layer[2];    // two layers
    layers[0]=new Layer(size_x*size_y, nb_layer1);  // nb_output[0] = nb_input[1]
    layers[1]=new Layer(nb_layer1, nb_layer2);
```

Artificial Intelligence for Robotics

- **Change the learning and evaluation process**
- The learning process must respect the following order:
  - Compute outputs from **input to output**
  - Computes deltas from **output to input** (and starting with setLastDeltas)
  - Update weights of layers (no specific order)

```
// dataset loop
for (test=0;test<matrixImages.length;test++){

    // define expected result
    int[] output=new int[nb_layer2];// vector of 0
    output[matrixLabels[test]]=1;   // answer at 1

    //Forward propagation
    layers[0].compute(matrixImages[test]);
    layers[1].compute(layers[0].results);

    // backward propagation (deltas)
    layers[1].setLastDeltas(output);
    layers[0].setDeltas(layers[1]);

    //weights update
    layers[0].learn(matrixImages[test]);
    layers[1].learn(layers[0].results);

    sumdelta+=layers[1].sum_delta; // add error of last layer

    display.repaint(); // update display panel
}
```

Artificial Intelligence for Robotics

- **Change the learning and evaluation process**

- For the evaluation, we only need the forward propagation:
  - Compute outputs from **input to output**
  - The error is obtained with output of last layer

```
// load evaluation dataset
dataset.setData("t10k-images.idx3-ubyte","t10k-labels.idx1-ubyte");


for (test=0;test<matrixImages.length;test++){

    //Forward propagation
    layers[0].compute(matrixImages[test]);
    layers[1].compute(layers[0].results);

    int imax=0;
    float max=0;
    for (int i=0;i<layers[1].results.length;i++){
        if (layers[1].results[i]>max){
            imax=i;
            max=layers[1].results[i];
        }
    }

    // count errors
    if (imax!=matrixLabels[test]) errors++;
}

// display number of errors
System.out.println("total "+errors+" errors");
```

Artificial Intelligence for Robotics

- **Change the learning and evaluation process**

- We finally update the DisplayFrame:
  - Code available on the github repository

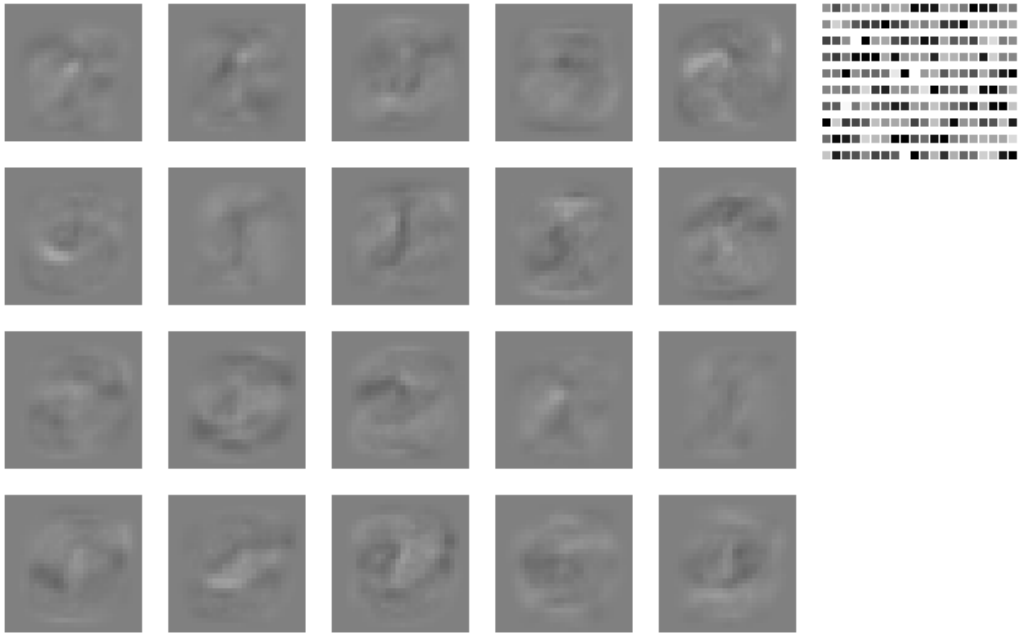
```
// weights of first layer
for (int n=0;n<Main.nb_layer1;n++){
    for (int i=0;i<Main.size_x;i++){
        for (int j=0;j<Main.size_y;j++){
            val=(int)(main.layers[0].weights[n][i+Main.size_x*j]*50)+128;
            if (val<0) val=0;
            if (val>255) val=255;
            g.setColor(new Color(val,val,val));
            g.fillRect(180+3*i + 100*(n%5), 10+3*j + 100*(n/5), 3, 3);
        }
    }
}

// weights of second layer
for(int n = 0; n < Main.nb_layer2; n++) {
    for (int i = 0; i < Main.nb_layer1 ; i++) {
        val = (int) (main.layers[1].weights[n][i] * 50) + 128;
        if (val < 0) val = 0;
        if (val > 255) val = 255;
        g.setColor(new Color(val, val, val));
        g.fillRect(680+6*i, 10+10*n, 5, 5);
    }
}
```

Artificial Intelligence for Robotics

- **Test your network**

- As we have far more weights to train, increase the epoch number to 100



```
number of labels is: 10000
total 839 errors
```



```
number of labels is: 10000
total 681 errors
```

- First layer defines features instead of average images
- Second layer combine features

- Many efforts for a slight improvement